

Wheelzy

Technical Assessment — Answers

Candidate: Orieb

Repository: <https://github.com/oriieebgSmile96/wheelzy-technical-assessment>

Solution: .NET 10, EF Core 10, MediatR, Roslyn, xUnit

Run tests: `dotnet test Wheelzy.Assessment.slnx`

Contents

1. Database design, SQL and Entity Framework
2. Caching rarely-changing data
3. UpdateCustomersBalanceByInvoices
4. GetOrders with optional filters
5. Processing .cs files + unit tests
6. Code review and corrections

Question 1 — Database design, SQL query, Entity Framework

A customer wants to sell a car. We store the car catalog once (make → model → submodel), the car's year and zip on the sale case, buyers who cover zips with a standard quote, one current quote per case, and a status history.

Why the tables are split this way

- **CarMake / CarModel / CarSubmodel** keep Honda, Civic, and EX as one row each. Many cases can point at the same submodel instead of repeating strings.
- **Year stays on SaleCase.** Year belongs to this specific vehicle, not to the catalog.
- **ZipCode** is its own table so buyers and cases share the same location list.
- **Buyer.QuoteAmount** is the standard offer (Buyer ABC covers 10 zips and pays \$500 for each car). **BuyerZipCode** is the coverage list.
- **CaseQuote** copies the amount onto the case. If the buyer later changes their rate, old cases keep the quoted number. **IsCurrent** marks the chosen quote. A filtered unique index guarantees only one current quote per case. Current is not required to be the highest amount.
- **CaseStatusType** includes **RequiresStatusDate**. **Picked Up** has that flag set. A trigger rejects a Picked Up row with a null **StatusDate**. Other statuses may leave **StatusDate** null.
- **CaseStatusHistory** stores every change (who, when) plus **IsCurrent** for the live status.
- Audit columns (**CreatedBy**, **ModifiedOn**, **ModifiedBy**, **IsActive**, **IsDeleted**) exist in SQL for every table whose entity inherits **BaseEntity**, so the EF model and the database match.

Schema: Wheelzy.Persistence/Sql/01-schema.sql

SQL query

Only the requested columns: car year/make/model/submodel, current buyer name and quote, current status name and status date.

```
SELECT
    c.Year,
    mk.Name AS Make,
    md.Name AS Model,
    sm.Name AS Submodel,
    b.Name AS CurrentBuyerName,
    q.Amount AS CurrentQuoteAmount,
    st.Name AS CurrentStatusName,
    h.StatusDate AS CurrentStatusDate
FROM dbo.SaleCase AS c
INNER JOIN dbo.CarSubmodel AS sm ON sm.SubmodelId = c.SubmodelId
INNER JOIN dbo.CarModel AS md ON md.ModelId = sm.ModelId
INNER JOIN dbo.CarMake AS mk ON mk.MakeId = md.MakeId
LEFT JOIN dbo.CaseQuote AS q
    ON q.CaseId = c.CaseId
    AND q.IsCurrent = 1
LEFT JOIN dbo.Buyer AS b ON b.BuyerId = q.BuyerId
LEFT JOIN dbo.CaseStatusHistory AS h
```

```
    ON h.CaseId = c.CaseId
    AND h.IsCurrent = 1
LEFT JOIN dbo.CaseStatusType AS st ON st.StatusTypeId = h.StatusTypeId;
```

Left joins are used so a new case still appears if a current quote or status has not been set yet.

Entity Framework

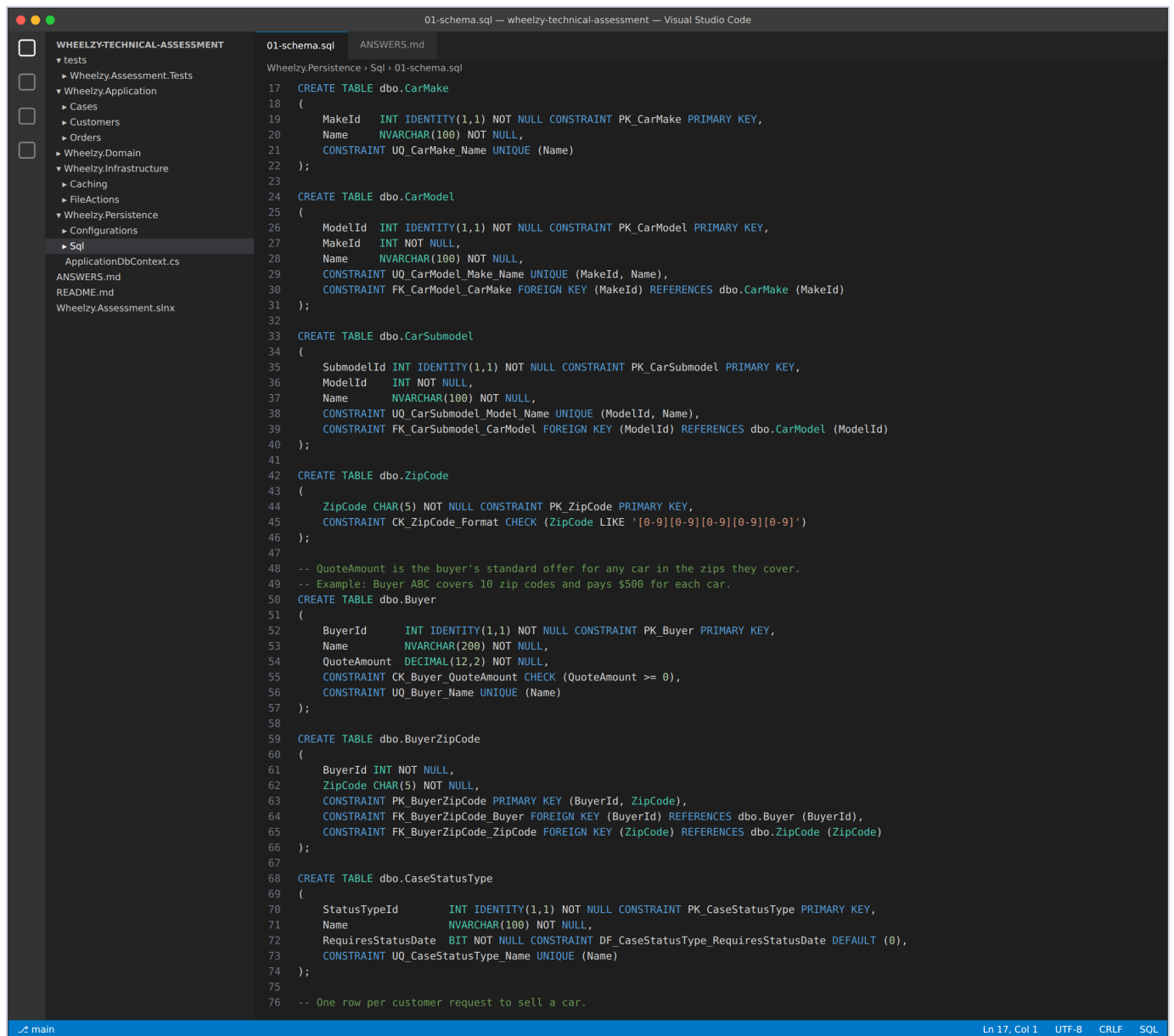
The same shape as the SQL (one LEFT JOIN per "current" row), projected to `CaseSummaryDto` so EF only selects the needed columns:

```
var query =
    from saleCase in dbContext.Cases.AsNoTracking()
    join currentQuote in dbContext.CaseQuotes.Where(q => q.IsCurrent)
      on saleCase.CaseId equals currentQuote.CaseId into quotes
    from currentQuote in quotes.DefaultIfEmpty()
    join currentStatus in dbContext.CaseStatusHistory.Where(s => s.IsCurrent)
      on saleCase.CaseId equals currentStatus.CaseId into statuses
    from currentStatus in statuses.DefaultIfEmpty()
    select new CaseSummaryDto
    {
        Year = saleCase.Year,
        Make = saleCase.Submodel.Model.Make.Name,
        Model = saleCase.Submodel.Model.Name,
        Submodel = saleCase.Submodel.Name,
        CurrentBuyerName = currentQuote == null ? null : currentQuote.Buyer.Name,
        CurrentQuoteAmount = currentQuote == null ? (decimal?)null : currentQuote.Amount,
        CurrentStatusName = currentStatus == null ? null : currentStatus.StatusType.Name,
        CurrentStatusDate = currentStatus == null ? (DateTime?)null : currentStatus.StatusDate
    };

return await query.ToListAsync(cancellationToken);
```

Code: `Wheelzy.Application/Cases/Queries/GetCaseSummaries/GetCaseSummariesQueryHandler.cs`

Screenshots from the code



```
01-schema.sql — wheelzy-technical-assessment — Visual Studio Code

Wheelzy.Persistence > Sql > 01-schema.sql

17 CREATE TABLE dbo.CarMake
18 (
19     MakeId INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_CarMake PRIMARY KEY,
20     Name NVARCHAR(100) NOT NULL,
21     CONSTRAINT UQ_CarMake_Name UNIQUE (Name)
22 );
23
24 CREATE TABLE dbo.CarModel
25 (
26     ModelId INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_CarModel PRIMARY KEY,
27     MakeId INT NOT NULL,
28     Name NVARCHAR(100) NOT NULL,
29     CONSTRAINT UQ_CarModel_Make_Name UNIQUE (MakeId, Name),
30     CONSTRAINT FK_CarModel_CarMake FOREIGN KEY (MakeId) REFERENCES dbo.CarMake (MakeId)
31 );
32
33 CREATE TABLE dbo.CarSubmodel
34 (
35     SubmodelId INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_CarSubmodel PRIMARY KEY,
36     ModelId INT NOT NULL,
37     Name NVARCHAR(100) NOT NULL,
38     CONSTRAINT UQ_CarSubmodel_Model_Name UNIQUE (ModelId, Name),
39     CONSTRAINT FK_CarSubmodel_CarModel FOREIGN KEY (ModelId) REFERENCES dbo.CarModel (ModelId)
40 );
41
42 CREATE TABLE dbo.ZipCode
43 (
44     ZipCode CHAR(5) NOT NULL CONSTRAINT PK_ZipCode PRIMARY KEY,
45     CONSTRAINT CK_ZipCode_Format CHECK (ZipCode LIKE '[0-9][0-9][0-9][0-9][0-9]')
46 );
47
48 -- QuoteAmount is the buyer's standard offer for any car in the zips they cover.
49 -- Example: Buyer ABC covers 10 zip codes and pays $500 for each car.
50 CREATE TABLE dbo.Buyer
51 (
52     BuyerId INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_Buyer PRIMARY KEY,
53     Name NVARCHAR(200) NOT NULL,
54     QuoteAmount DECIMAL(12,2) NOT NULL,
55     CONSTRAINT CK_Buyer_QuoteAmount CHECK (QuoteAmount >= 0),
56     CONSTRAINT UQ_Buyer_Name UNIQUE (Name)
57 );
58
59 CREATE TABLE dbo.BuyerZipCode
60 (
61     BuyerId INT NOT NULL,
62     ZipCode CHAR(5) NOT NULL,
63     CONSTRAINT PK_BuyerZipCode PRIMARY KEY (BuyerId, ZipCode),
64     CONSTRAINT FK_BuyerZipCode_Buyer FOREIGN KEY (BuyerId) REFERENCES dbo.Buyer (BuyerId),
65     CONSTRAINT FK_BuyerZipCode_ZipCode FOREIGN KEY (ZipCode) REFERENCES dbo.ZipCode (ZipCode)
66 );
67
68 CREATE TABLE dbo.CaseStatusType
69 (
70     StatusTypeId INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_CaseStatusType PRIMARY KEY,
71     Name NVARCHAR(100) NOT NULL,
72     RequiresStatusDate BIT NOT NULL CONSTRAINT DF_CaseStatusType_RequiresStatusDate DEFAULT (0),
73     CONSTRAINT UQ_CaseStatusType_Name UNIQUE (Name)
74 );
75
76 -- One row per customer request to sell a car.
```

01-schema.sql — catalog, zip codes, buyers, status types

01-schema.sql — wheelzy-technical-assessment — Visual Studio Code

WHEELZY-TECHNICAL-ASSESSMENT

tests

Wheelzy.Assessment.Tests

Wheelzy.Application

Cases

Customers

Orders

Wheelzy.Domain

Wheelzy.Infrastructure

Caching

FileActions

Wheelzy.Persistence

Configurations

Sql

ApplicationDbContext.cs

ANSWERS.md

README.md

Wheelzy.Assessment.slnx

01-schema.sql

ANSWERS.md

Wheelzy.Persistence > Sql > 01-schema.sql

77 -- Year stays on the case because it belongs to this specific vehicle, not the catalog.
78 CREATE TABLE dbo.SaleCase
79 (
80 CaseId INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_SaleCase PRIMARY KEY,
81 Year SMALLINT NOT NULL,
82 SubmodelId INT NOT NULL,
83 ZipCode CHAR(5) NOT NULL,
84 CreatedAt DATETIME2(0) NOT NULL CONSTRAINT DF_SaleCase_CreatedAt DEFAULT (SYSUTCDATETIME()),
85 CreatedBy NVARCHAR(256) NULL,
86 ModifiedOn DATETIME2(0) NULL,
87 ModifiedBy NVARCHAR(256) NULL,
88 IsActive BIT NOT NULL CONSTRAINT DF_SaleCase_IsActive DEFAULT (1),
89 IsDeleted BIT NOT NULL CONSTRAINT DF_SaleCase_IsDeleted DEFAULT (0),
90 CONSTRAINT CK_SaleCase_Year CHECK (Year BETWEEN 1900 AND 2100),
91 CONSTRAINT FK_SaleCase_CarSubmodel FOREIGN KEY (SubmodelId) REFERENCES dbo.CarSubmodel (SubmodelId),
92 CONSTRAINT FK_SaleCase_ZipCode FOREIGN KEY (ZipCode) REFERENCES dbo.ZipCode (ZipCode)
93);
94
95 -- Quotes are snapshotted onto the case so a later change to Buyer.QuoteAmount
96 -- does not rewrite history. Exactly one quote can be current per case.
97 CREATE TABLE dbo.CaseQuote
98 (
99 CaseQuoteId INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_CaseQuote PRIMARY KEY,
100 CaseId INT NOT NULL,
101 BuyerId INT NOT NULL,
102 Amount DECIMAL(12,2) NOT NULL,
103 IsCurrent BIT NOT NULL CONSTRAINT DF_CaseQuote_IsCurrent DEFAULT (0),
104 CreatedAt DATETIME2(0) NOT NULL CONSTRAINT DF_CaseQuote_CreatedAt DEFAULT (SYSUTCDATETIME()),
105 CreatedBy NVARCHAR(256) NULL,
106 ModifiedOn DATETIME2(0) NULL,
107 ModifiedBy NVARCHAR(256) NULL,
108 IsActive BIT NOT NULL CONSTRAINT DF_CaseQuote_IsActive DEFAULT (1),
109 IsDeleted BIT NOT NULL CONSTRAINT DF_CaseQuote_IsDeleted DEFAULT (0),
110 CONSTRAINT CK_CaseQuote_Amount CHECK (Amount >= 0),
111 CONSTRAINT UQ_CaseQuote_Case_Buyer UNIQUE (CaseId, BuyerId),
112 CONSTRAINT FK_CaseQuote_SaleCase FOREIGN KEY (CaseId) REFERENCES dbo.SaleCase (CaseId),
113 CONSTRAINT FK_CaseQuote_Buyer FOREIGN KEY (BuyerId) REFERENCES dbo.Buyer (BuyerId)
114);
115
116 CREATE UNIQUE INDEX UX_CaseQuote_Current
117 ON dbo.CaseQuote (CaseId)
118 WHERE IsCurrent = 1;
119
120 CREATE TABLE dbo.CaseStatusHistory
121 (
122 CaseStatusId INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_CaseStatusHistory PRIMARY KEY,
123 CaseId INT NOT NULL,
124 StatusTypeId INT NOT NULL,
125 IsCurrent BIT NOT NULL CONSTRAINT DF_CaseStatusHistory_IsCurrent DEFAULT (0),
126 ChangedBy NVARCHAR(256) NOT NULL,
127 ChangedAt DATETIME2(0) NOT NULL CONSTRAINT DF_CaseStatusHistory_ChangedAt DEFAULT (SYSUTCDATETIME()),
128 -- Mandatory only when the status type requires it (Picked Up). Enforced below.
129 StatusDate DATETIME2(0) NULL,
130 -- Audit columns shared by every BaseEntity (mapped by EF Core).
131 CreatedAt DATETIME2(0) NOT NULL CONSTRAINT DF_CaseStatusHistory_CreatedAt DEFAULT (SYSUTCDATETIME()),
132 CreatedBy NVARCHAR(256) NULL,
133 ModifiedOn DATETIME2(0) NULL,
134 ModifiedBy NVARCHAR(256) NULL,
135 IsActive BIT NOT NULL CONSTRAINT DF_CaseStatusHistory_IsActive DEFAULT (1),
136 IsDeleted BIT NOT NULL CONSTRAINT DF_CaseStatusHistory_IsDeleted DEFAULT (0),
137 CONSTRAINT FK_CaseStatusHistory_SaleCase FOREIGN KEY (CaseId) REFERENCES dbo.SaleCase (CaseId),
138 CONSTRAINT FK_CaseStatusHistory_CaseStatusType FOREIGN KEY (StatusTypeId) REFERENCES dbo.CaseStatusType (StatusTypeId)
139);
140

main Ln 77, Col 1 UTF-8 CRLF SQL

01-schema.sql — SaleCase, CaseQuote (one current quote), CaseStatusHistory

OrieB — Wheelzy Technical Assessment — 5/24

```
01-schema.sql — wheelzy-technical-assessment — Visual Studio Code

Wheelzy.Persistence > Sql > 01-schema.sql

140
141 CREATE UNIQUE INDEX UX_CaseStatusHistory_Current
142 ON dbo.CaseStatusHistory (CaseId)
143 WHERE IsCurrent = 1;
144
145 -- Supports reading the full status timeline of a case in order.
146 CREATE INDEX IX_CaseStatusHistory_Case_ChangedAt
147 ON dbo.CaseStatusHistory (CaseId, ChangedAt);
148
149 GO
150
151 CREATE OR ALTER TRIGGER dbo.TR_CaseStatusHistory_RequireStatusDate
152 ON dbo.CaseStatusHistory
153 AFTER INSERT, UPDATE
154 AS
155 BEGIN
156     SET NOCOUNT ON;
157
158     IF EXISTS
159     (
160         SELECT 1
161         FROM inserted i
162         INNER JOIN dbo.CaseStatusType t ON t.StatusTypeId = i.StatusTypeId
163         WHERE t.RequiresStatusDate = 1
164             AND i.StatusDate IS NULL
165     )
166     BEGIN
167         THROW 50001, 'StatusDate is required when the status type requires a status date (for example Picked Up).', 1;
168     END
169 END;
170 GO
171
172 INSERT INTO dbo.CaseStatusType (Name, RequiresStatusDate)
173 VALUES
174     ('Pending Acceptance', 0),
175     ('Accepted', 0),
176     ('Picked Up', 1);
177 GO
```

01-schema.sql — indexes, Picked Up trigger, seed data

```
02-case-summary.sql — wheelzy-technical-assessment — Visual Studio Code

Wheelzy.Persistence > Sql > 02-case-summary.sql

1 -- Question 1: car info, current buyer + quote, current status + status date.
2 -- Only the columns requested. Current quote is not assumed to be the highest.
3
4 SELECT
5     c.Year,
6     mk.Name AS Make,
7     md.Name AS Model,
8     sm.Name AS Submodel,
9     b.Name AS CurrentBuyerName,
10    q.Amount AS CurrentQuoteAmount,
11    st.Name AS CurrentStatusName,
12    h.StatusDate AS CurrentStatusDate
13 FROM dbo.SaleCase AS c
14 INNER JOIN dbo.CarSubmodel AS sm ON sm.SubmodelId = c.SubmodelId
15 INNER JOIN dbo.CarModel AS md ON md.ModelId = sm.ModelId
16 INNER JOIN dbo.CarMake AS mk ON mk.MakeId = md.MakeId
17 LEFT JOIN dbo.CaseQuote AS q
18     ON q.CaseId = c.CaseId
19     AND q.IsCurrent = 1
20 LEFT JOIN dbo.Buyer AS b ON b.BuyerId = q.BuyerId
21 LEFT JOIN dbo.CaseStatusHistory AS h
22     ON h.CaseId = c.CaseId
23     AND h.IsCurrent = 1
24 LEFT JOIN dbo.CaseStatusType AS st ON st.StatusTypeId = h.StatusTypeId;
```

02-case-summary.sql — the SQL query

```
GetCaseSummariesQueryHandler.cs — wheelzy-technical-assessment — Visual Studio Code

WHEELZY-TECHNICAL-ASSESSMENT
├── tests
│   ├── Wheelzy.Assessment.Tests
│   └── Wheelzy.Application
│       ├── Cases
│       ├── Customers
│       ├── Orders
│       ├── Wheelzy.Domain
│       ├── Wheelzy.Infrastructure
│       ├── Caching
│       ├── FileActions
│       ├── Wheelzy.Persistence
│       ├── Configurations
│       ├── Sql
│       ├── ApplicationDbContext.cs
│       ├── ANSWERS.md
│       ├── README.md
│       └── Wheelzy.Assessment.slnx
└── Wheelzy.Application.Cases.Queries.GetCaseSummaries
    └── GetCaseSummariesQueryHandler.cs

1  using MediatR;
2  using Microsoft.EntityFrameworkCore;
3  using Wheelzy.Application.Cases.Queries.GetCaseSummaries.Dtos;
4  using Wheelzy.Application.Common.Interfaces;
5
6  namespace Wheelzy.Application.Cases.Queries.GetCaseSummaries;
7
8  public class GetCaseSummariesQueryHandler : IRequestHandler<GetCaseSummariesQuery, List<CaseSummaryDto>>
9  {
10     private readonly IApplicationDbContext _applicationDbContext;
11
12     public GetCaseSummariesQueryHandler(IApplicationDbContext applicationDbContext)
13     {
14         _applicationDbContext = applicationDbContext;
15     }
16
17     public Task<List<CaseSummaryDto>> Handle(GetCaseSummariesQuery request, CancellationToken cancellationToken)
18     {
19         // One LEFT JOIN per "current" row (same shape as the SQL query) instead of
20         // four correlated sub-queries (one per projected column).
21         var query =
22             from saleCase in _applicationDbContext.Cases.AsNoTracking()
23             join currentQuote in _applicationDbContext.CaseQuotes.Where(q => q.IsCurrent)
24               on saleCase.CaseId equals currentQuote.CaseId into quotes
25             from currentQuote in quotes.DefaultIfEmpty()
26             join currentStatus in _applicationDbContext.CaseStatusHistory.Where(s => s.IsCurrent)
27               on saleCase.CaseId equals currentStatus.CaseId into statuses
28             from currentStatus in statuses.DefaultIfEmpty()
29             select new CaseSummaryDto
30             {
31                 Year = saleCase.Year,
32                 Make = saleCase.Submodel.Model.Make.Name,
33                 Model = saleCase.Submodel.Model.Name,
34                 Submodel = saleCase.Submodel.Name,
35                 CurrentBuyerName = currentQuote == null ? null : currentQuote.Buyer.Name,
36                 CurrentQuoteAmount = currentQuote == null ? (decimal?)null : currentQuote.Amount,
37                 CurrentStatusName = currentStatus == null ? null : currentStatus.StatusType.Name,
38                 CurrentStatusDate = currentStatus == null ? (DateTime?)null : currentStatus.StatusDate
39             };
40
41         return query
42             .ToListAsync(cancellationToken);
43     }
44 }
```

GetCaseSummariesQueryHandler.cs — the Entity Framework query

Question 2 — Data that rarely changes but is read all the time

Cache it. Good examples in this domain: makes/models/submodels, zip coverage, status types, buyer standard quotes.

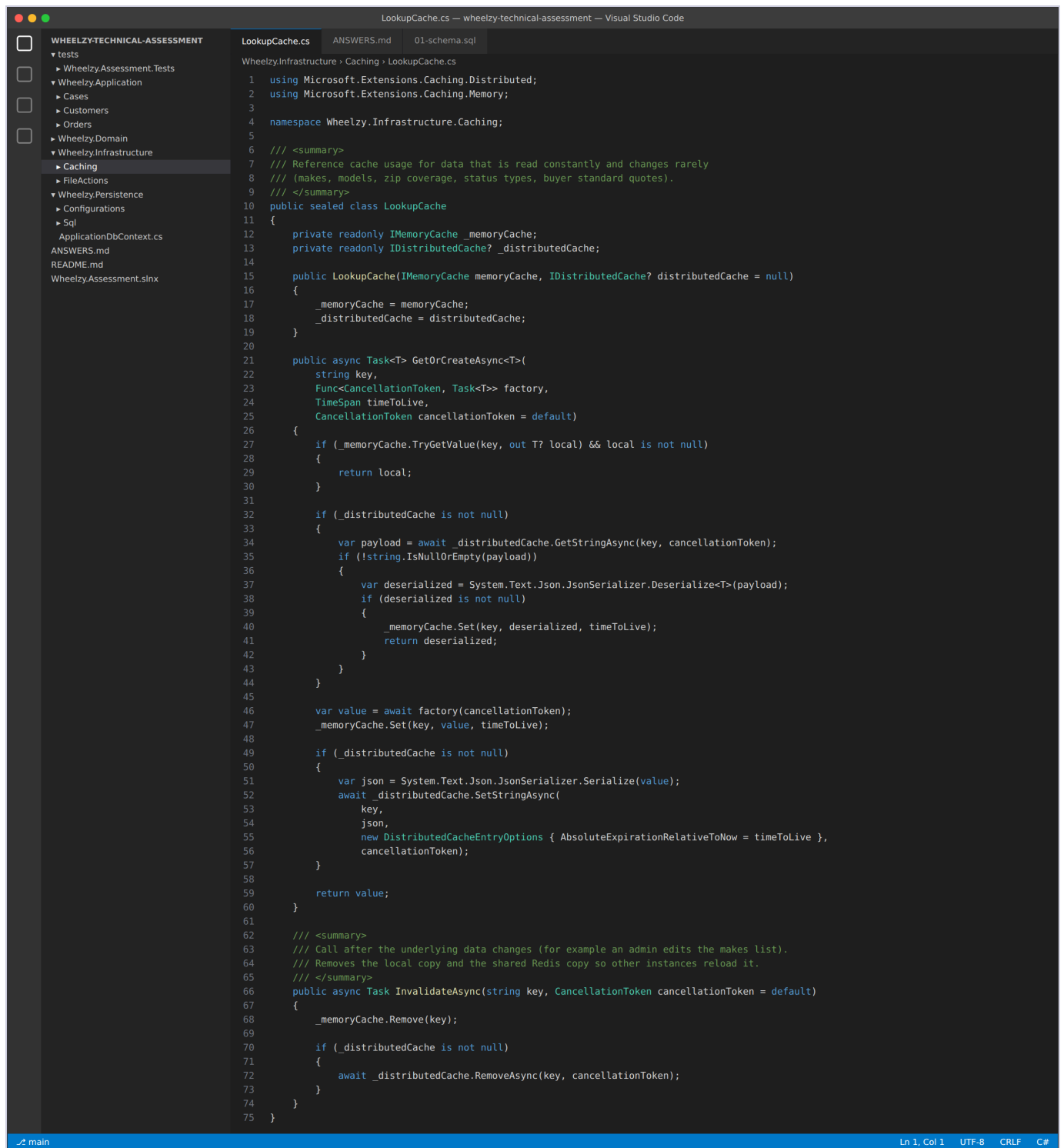
Single instance: IMemoryCache (or a static lookup loaded at startup) is enough. Use a sliding or long absolute expiration, and invalidate when an admin updates the lookup.

More than one instance: yes, it matters.

- In-memory cache is per process. Instance A can hold a stale make list after Instance B updates SQL Server.
- For multiple instances I would keep a short in-memory cache in front of a **distributed cache** (Redis / IDistributedCache). Writes update the database, then remove or overwrite the Redis key. Optionally publish an invalidation message so other nodes drop their local copy.
- If the data is tiny and almost never changes, loading it once at startup and recycling the app (or pushing a refresh signal) is also valid.

Reference helper: Wheelzy.Infrastructure/Caching/LookupCache.cs (GetOrCreateAsync + InvalidateAsync, which clears both the local and the Redis copy).

Screenshots from the code



```
LookupCache.cs — wheelzy-technical-assessment — Visual Studio Code

Wheelzy.Infrastructure > Caching > LookupCache.cs
1  using Microsoft.Extensions.Caching.Distributed;
2  using Microsoft.Extensions.Caching.Memory;
3
4  namespace Wheelzy.Infrastructure.Caching;
5
6  /// <summary>
7  /// Reference cache usage for data that is read constantly and changes rarely
8  /// (makes, models, zip coverage, status types, buyer standard quotes).
9  /// </summary>
10 public sealed class LookupCache
11 {
12     private readonly IMemoryCache _memoryCache;
13     private readonly IDistributedCache? _distributedCache;
14
15     public LookupCache(IMemoryCache memoryCache, IDistributedCache? distributedCache = null)
16     {
17         _memoryCache = memoryCache;
18         _distributedCache = distributedCache;
19     }
20
21     public async Task<T> GetOrCreateAsync<T>(
22         string key,
23         Func<CancellationToken, Task<T>> factory,
24         TimeSpan timeToLive,
25         CancellationToken cancellationToken = default)
26     {
27         if (_memoryCache.TryGetValue(key, out T? local) && local is not null)
28         {
29             return local;
30         }
31
32         if (_distributedCache is not null)
33         {
34             var payload = await _distributedCache.GetStringAsync(key, cancellationToken);
35             if (!string.IsNullOrEmpty(payload))
36             {
37                 var deserialized = System.Text.Json.JsonSerializer.Deserialize<T>(payload);
38                 if (deserialized is not null)
39                 {
40                     _memoryCache.Set(key, deserialized, timeToLive);
41                     return deserialized;
42                 }
43             }
44         }
45
46         var value = await factory(cancellationToken);
47         _memoryCache.Set(key, value, timeToLive);
48
49         if (_distributedCache is not null)
50         {
51             var json = System.Text.Json.JsonSerializer.Serialize(value);
52             await _distributedCache.SetStringAsync(
53                 key,
54                 json,
55                 new DistributedCacheEntryOptions { AbsoluteExpirationRelativeToNow = timeToLive },
56                 cancellationToken);
57         }
58
59         return value;
60     }
61
62     /// <summary>
63     /// Call after the underlying data changes (for example an admin edits the makes list).
64     /// Removes the local copy and the shared Redis copy so other instances reload it.
65     /// </summary>
66     public async Task InvalidateAsync(string key, CancellationToken cancellationToken = default)
67     {
68         _memoryCache.Remove(key);
69
70         if (_distributedCache is not null)
71         {
72             await _distributedCache.RemoveAsync(key, cancellationToken);
73         }
74     }
75 }
```

LookupCache.cs — memory cache in front of a distributed (Redis) cache

Question 3 — UpdateCustomersBalanceByInvoices

Original method problems:

1. `SingleOrDefault(invoice.CustomerId.Value)` is not a predicate, so it does not compile. It should be `c => c.Id == invoice.CustomerId`.
2. One query per invoice (N+1).
3. `SaveChanges()` inside the loop = one transaction per invoice.
4. Missing customer becomes a `NullReferenceException`.
5. `CustomerId.Value` throws if `CustomerId` is null.
6. Two invoices for the same customer should sum, not apply one at a time with extra round-trips.

Rewritten approach:

- Ignore null/empty invoice lists and invoices with no customer.
- Group totals by customer id in memory.
- Load those customers in **one** query.
- Subtract the grouped total from each customer.
- Call `SaveChanges` **once**.

```
public async Task UpdateCustomersBalanceByInvoicesAsync(
    IReadOnlyCollection<Invoice> invoices,
    CancellationToken cancellationToken = default)
{
    ArgumentNullException.ThrowIfNull(invoices);
    if (invoices.Count == 0) return;

    var totalsByCustomer = invoices
        .Where(invoice => invoice.CustomerId.HasValue)
        .GroupBy(invoice => invoice.CustomerId!.Value)
        .ToDictionary(group => group.Key, group => group.Sum(invoice => invoice.Total));

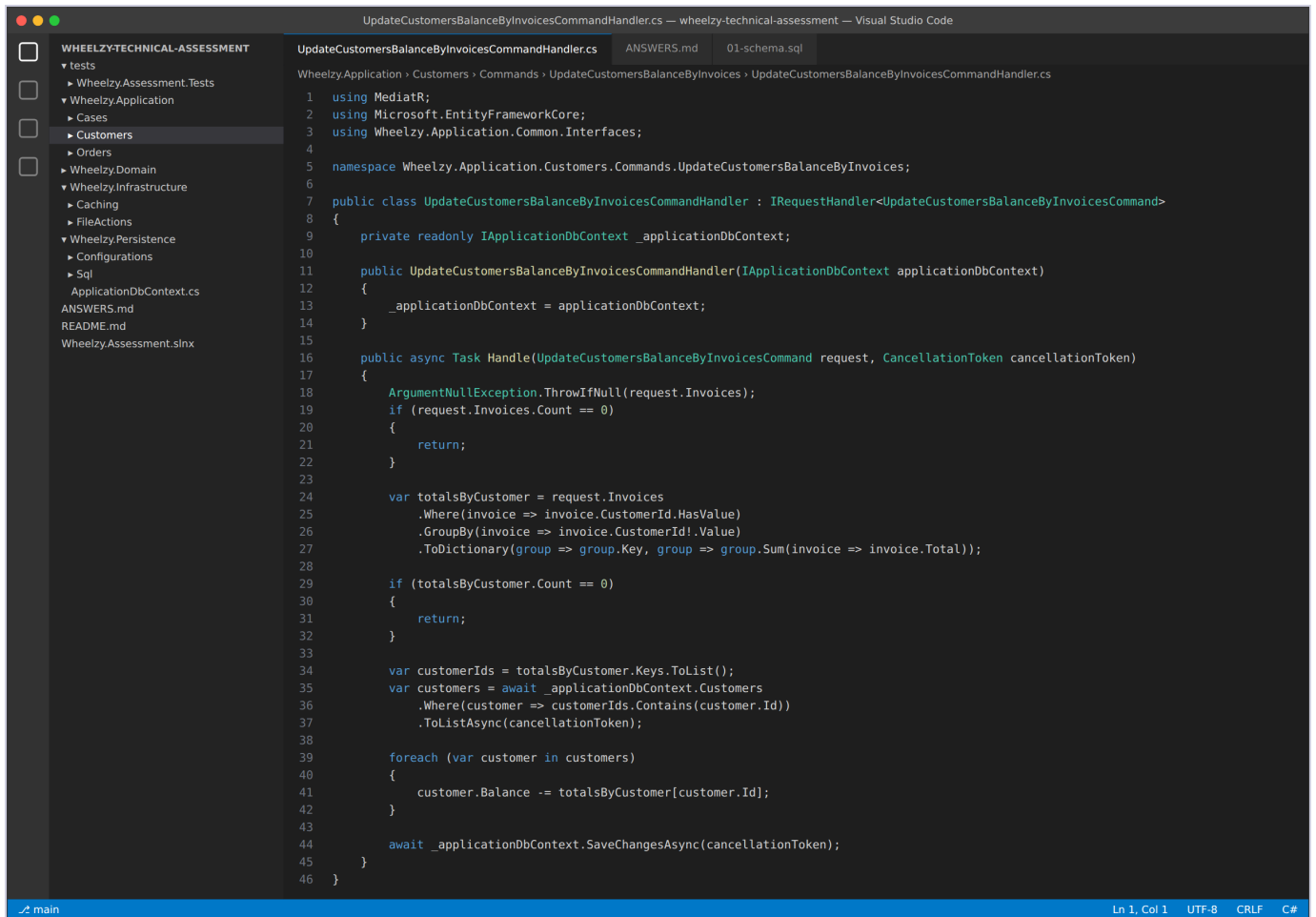
    if (totalsByCustomer.Count == 0) return;

    var customerIds = totalsByCustomer.Keys.ToList();
    var customers = await dbContext.Customers
        .Where(customer => customerIds.Contains(customer.Id))
        .ToListAsync(cancellationToken);

    foreach (var customer in customers)
        customer.Balance -= totalsByCustomer[customer.Id];

    await dbContext.SaveChangesAsync(cancellationToken);
}
```

Screenshots from the code



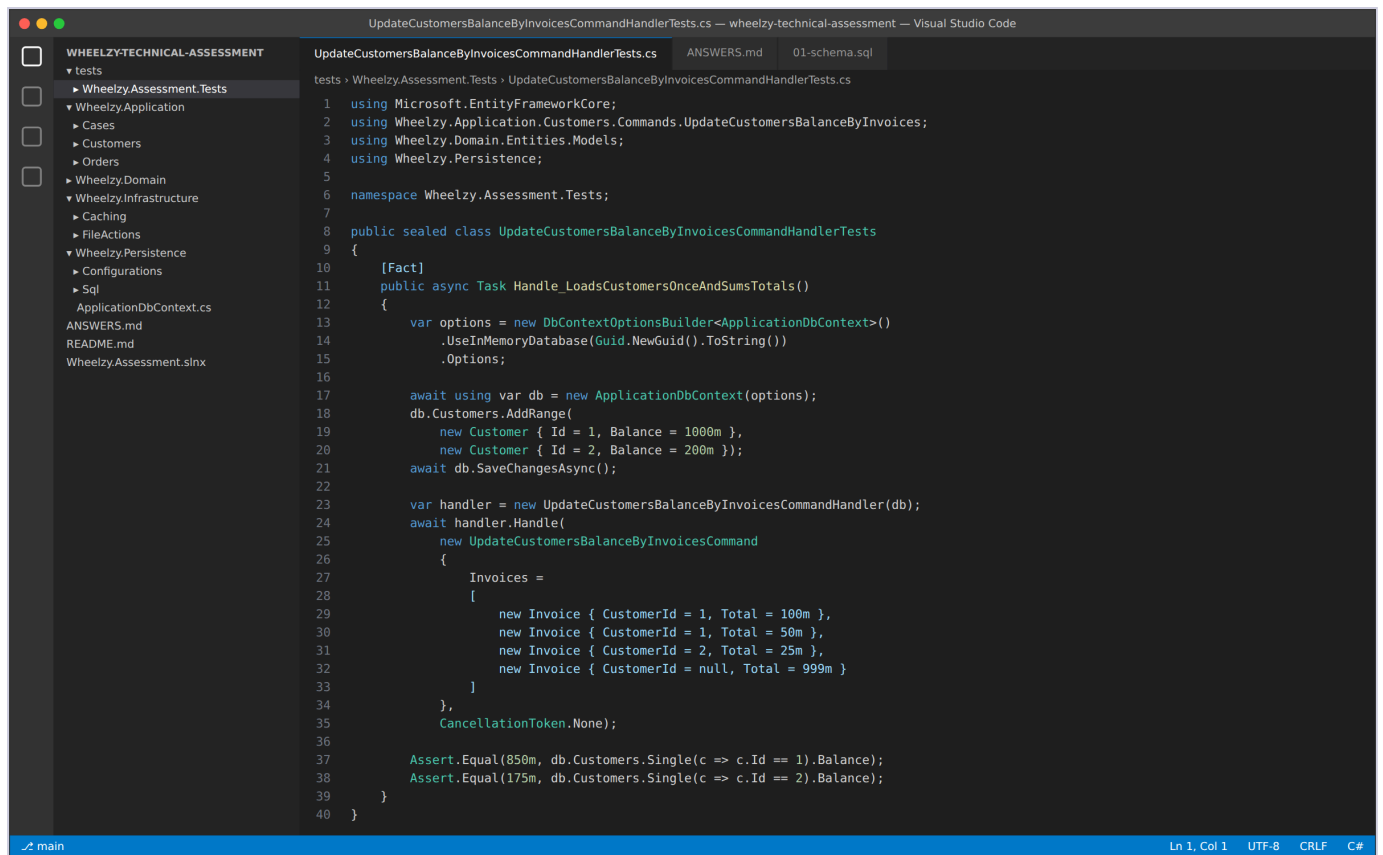
```
UpdateCustomersBalanceByInvoicesCommandHandler.cs — wheelzy-technical-assessment — Visual Studio Code

UpdateCustomersBalanceByInvoicesCommandHandler.cs  ANSWERS.md  01-schema.sql

Wheelzy.Application > Customers > Commands > UpdateCustomersBalanceByInvoices > UpdateCustomersBalanceByInvoicesCommandHandler.cs

1  using MediatR;
2  using Microsoft.EntityFrameworkCore;
3  using Wheelzy.Application.Common.Interfaces;
4
5  namespace Wheelzy.Application.Customers.Commands.UpdateCustomersBalanceByInvoices;
6
7  public class UpdateCustomersBalanceByInvoicesCommandHandler : IRequestHandler<UpdateCustomersBalanceByInvoicesCommand>
8  {
9      private readonly IApplicationDbContext _applicationDbContext;
10
11     public UpdateCustomersBalanceByInvoicesCommandHandler(IApplicationDbContext applicationDbContext)
12     {
13         _applicationDbContext = applicationDbContext;
14     }
15
16     public async Task Handle(UpdateCustomersBalanceByInvoicesCommand request, CancellationToken cancellationToken)
17     {
18         ArgumentNullException.ThrowIfNull(request.Invoices);
19         if (request.Invoices.Count == 0)
20         {
21             return;
22         }
23
24         var totalsByCustomer = request.Invoices
25             .Where(invoice => invoice.CustomerId.HasValue)
26             .GroupBy(invoice => invoice.CustomerId!.Value)
27             .ToDictionary(group => group.Key, group => group.Sum(invoice => invoice.Total));
28
29         if (totalsByCustomer.Count == 0)
30         {
31             return;
32         }
33
34         var customerIds = totalsByCustomer.Keys.ToList();
35         var customers = await _applicationDbContext.Customers
36             .Where(customer => customerIds.Contains(customer.Id))
37             .ToListAsync(cancellationToken);
38
39         foreach (var customer in customers)
40         {
41             customer.Balance -= totalsByCustomer[customer.Id];
42         }
43
44         await _applicationDbContext.SaveChangesAsync(cancellationToken);
45     }
46 }
```

UpdateCustomersBalanceByInvoicesCommandHandler.cs



```
UpdateCustomersBalanceByInvoicesCommandHandlerTests.cs — wheelzy-technical-assessment — Visual Studio Code

UpdateCustomersBalanceByInvoicesCommandHandlerTests.cs  ANSWERS.md  01-schema.sql

tests > Wheelzy.Assessment.Tests > UpdateCustomersBalanceByInvoicesCommandHandlerTests.cs
1  using Microsoft.EntityFrameworkCore;
2  using Wheelzy.Application.Customers.Commands.UpdateCustomersBalanceByInvoices;
3  using Wheelzy.Domain.Entities.Models;
4  using Wheelzy.Persistence;
5
6  namespace Wheelzy.Assessment.Tests;
7
8  public sealed class UpdateCustomersBalanceByInvoicesCommandHandlerTests
9  {
10     [Fact]
11     public async Task Handle_LoadsCustomersOnceAndSumsTotals()
12     {
13         var options = new DbContextOptionsBuilder<ApplicationDbContext>()
14             .UseInMemoryDatabase(Guid.NewGuid().ToString())
15             .Options;
16
17         await using var db = new ApplicationDbContext(options);
18         db.Customers.AddRange(
19             new Customer { Id = 1, Balance = 1000m },
20             new Customer { Id = 2, Balance = 200m });
21         await db.SaveChangesAsync();
22
23         var handler = new UpdateCustomersBalanceByInvoicesCommandHandler(db);
24         await handler.Handle(
25             new UpdateCustomersBalanceByInvoicesCommand
26             {
27                 Invoices =
28                 [
29                     new Invoice { CustomerId = 1, Total = 100m },
30                     new Invoice { CustomerId = 1, Total = 50m },
31                     new Invoice { CustomerId = 2, Total = 25m },
32                     new Invoice { CustomerId = null, Total = 999m }
33                 ],
34                 CancellationToken.None);
35
36         Assert.Equal(850m, db.Customers.Single(c => c.Id == 1).Balance);
37         Assert.Equal(175m, db.Customers.Single(c => c.Id == 2).Balance);
38     }
39 }
40 }
```

Unit test for Question 3

Question 4 — GetOrders with optional filters

Keep one `IQueryable<Order>` and add a `Where` only when that filter is set. Null or empty id lists mean “do not filter by that field,” as the prompt says.

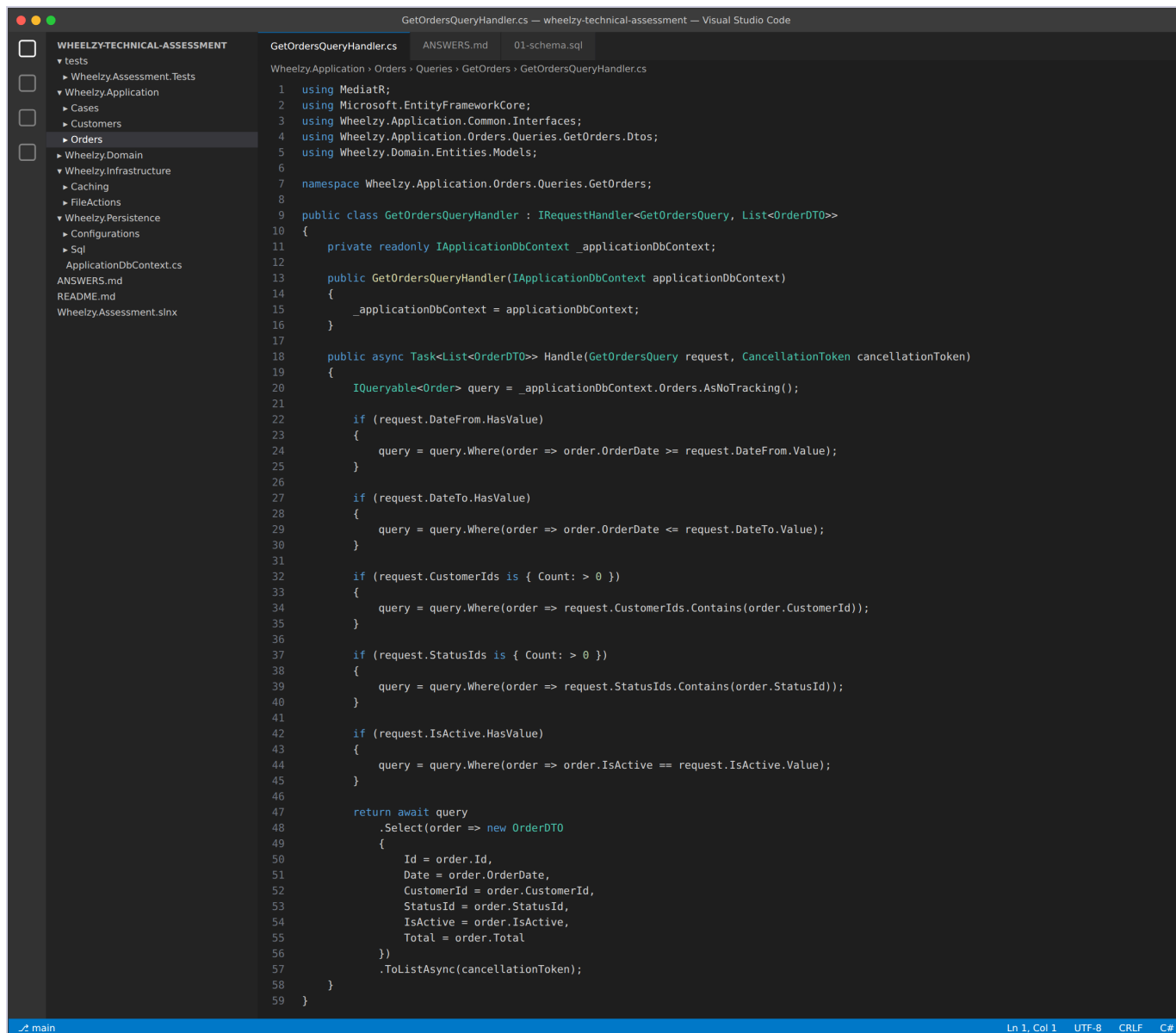
```
public async Task<List<OrderDTO>> GetOrders(
    DateTime? dateFrom, DateTime? dateTo,
    List<int> customerIds, List<int> statusIds, bool? isActive)
{
    IQueryable<Order> query = dbContext.Orders.AsNoTracking();

    if (dateFrom.HasValue)
        query = query.Where(o => o.OrderDate >= dateFrom.Value);
    if (dateTo.HasValue)
        query = query.Where(o => o.OrderDate <= dateTo.Value);
    if (customerIds is { Count: > 0 })
        query = query.Where(o => customerIds.Contains(o.CustomerId));
    if (statusIds is { Count: > 0 })
        query = query.Where(o => statusIds.Contains(o.StatusId));
    if (isActive.HasValue)
        query = query.Where(o => o.IsActive == isActive.Value);

    return await query
        .Select(o => new OrderDTO
        {
            Id = o.Id,
            Date = o.OrderDate,
            CustomerId = o.CustomerId,
            StatusId = o.StatusId,
            IsActive = o.IsActive,
            Total = o.Total
        })
        .ToListAsync();
}
```

This stays efficient when every filter is set, when some are set, and when none are set: unused filters never appear in the SQL. `AsNoTracking` plus a DTO projection avoids tracking and extra columns.

Screenshots from the code

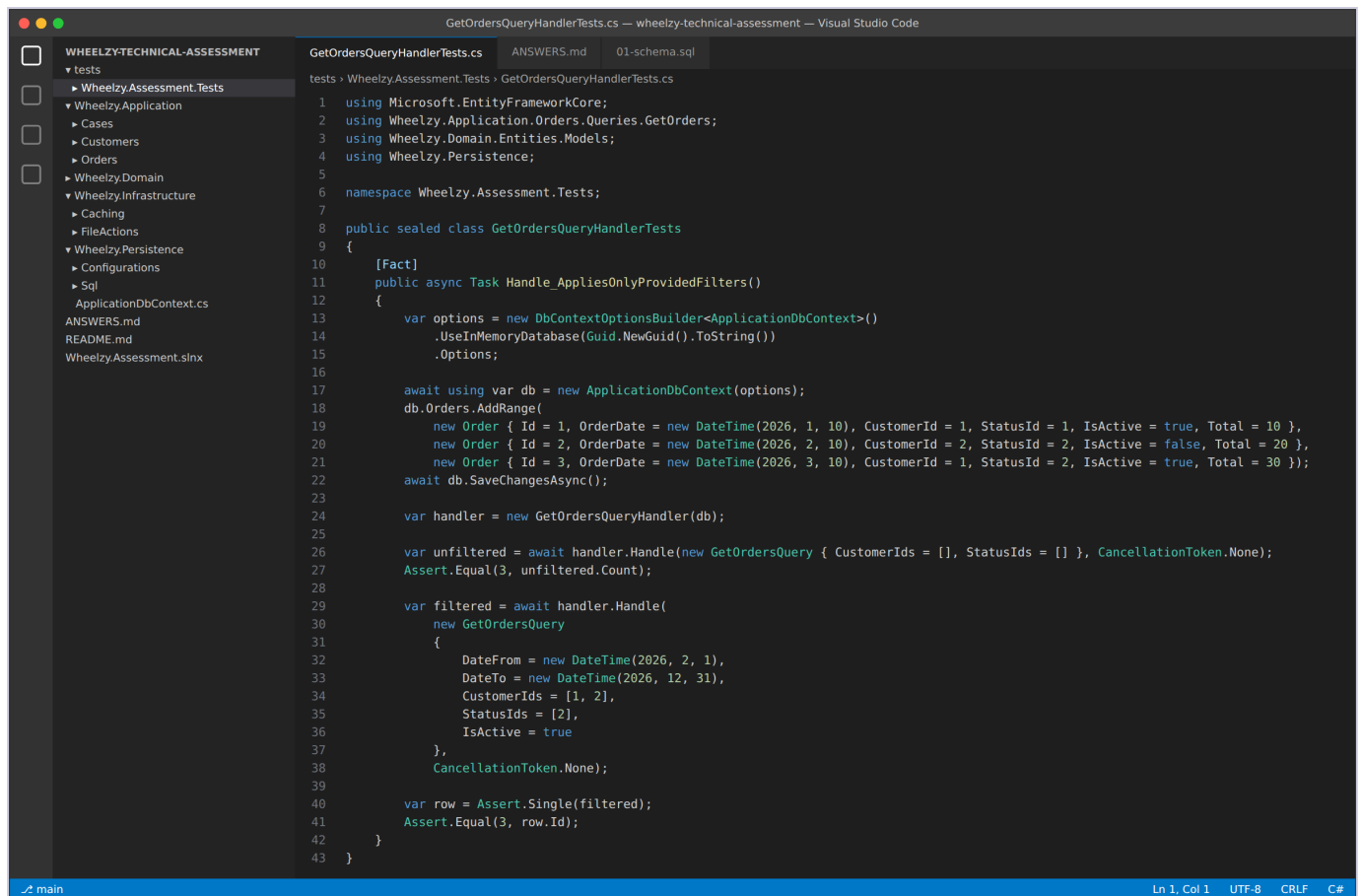


```
GetOrdersQueryHandler.cs — wheelzy-technical-assessment — Visual Studio Code

Wheelzy.Application > Orders > Queries > GetOrders > GetOrdersQueryHandler.cs

1  using MediatR;
2  using Microsoft.EntityFrameworkCore;
3  using Wheelzy.Application.Common.Interfaces;
4  using Wheelzy.Application.Orders.Queries.GetOrders.Dtos;
5  using Wheelzy.Domain.Entities.Models;
6
7  namespace Wheelzy.Application.Orders.Queries.GetOrders;
8
9  public class GetOrdersQueryHandler : IRequestHandler<GetOrdersQuery, List<OrderDTO>>
10 {
11     private readonly IApplicationDbContext _applicationDbContext;
12
13     public GetOrdersQueryHandler(IApplicationDbContext applicationDbContext)
14     {
15         _applicationDbContext = applicationDbContext;
16     }
17
18     public async Task<List<OrderDTO>> Handle(GetOrdersQuery request, CancellationToken cancellationToken)
19     {
20         IQueryable<Order> query = _applicationDbContext.Orders.AsNoTracking();
21
22         if (request.DateFrom.HasValue)
23         {
24             query = query.Where(order => order.OrderDate >= request.DateFrom.Value);
25         }
26
27         if (request.DateTo.HasValue)
28         {
29             query = query.Where(order => order.OrderDate <= request.DateTo.Value);
30         }
31
32         if (request.CustomerIds is { Count: > 0 })
33         {
34             query = query.Where(order => request.CustomerIds.Contains(order.CustomerId));
35         }
36
37         if (request.StatusIds is { Count: > 0 })
38         {
39             query = query.Where(order => request.StatusIds.Contains(order.StatusId));
40         }
41
42         if (request.IsActive.HasValue)
43         {
44             query = query.Where(order => order.IsActive == request.IsActive.Value);
45         }
46
47         return await query
48             .Select(order => new OrderDTO
49             {
50                 Id = order.Id,
51                 Date = order.OrderDate,
52                 CustomerId = order.CustomerId,
53                 StatusId = order.StatusId,
54                 IsActive = order.IsActive,
55                 Total = order.Total
56             })
57             .ToListAsync(cancellationToken);
58     }
59 }
```

GetOrdersQueryHandler.cs — filters added only when set



```
GetOrdersQueryHandlerTests.cs — wheelzy-technical-assessment — Visual Studio Code
GetOrdersQueryHandlerTests.cs  ANSWERS.md  01-schema.sql
tests > Wheelzy.Assessment.Tests > GetOrdersQueryHandlerTests.cs
1  using Microsoft.EntityFrameworkCore;
2  using Wheelzy.Application.Orders.Queries.GetOrders;
3  using Wheelzy.Domain.Entities.Models;
4  using Wheelzy.Persistence;
5
6  namespace Wheelzy.Assessment.Tests;
7
8  public sealed class GetOrdersQueryHandlerTests
9  {
10     [Fact]
11     public async Task Handle_AppliesOnlyProvidedFilters()
12     {
13         var options = new DbContextOptionsBuilder<ApplicationDbContext>()
14             .UseInMemoryDatabase(Guid.NewGuid().ToString())
15             .Options;
16
17         await using var db = new ApplicationDbContext(options);
18         db.Orders.AddRange(
19             new Order { Id = 1, OrderDate = new DateTime(2026, 1, 10), CustomerId = 1, StatusId = 1, IsActive = true, Total = 10 },
20             new Order { Id = 2, OrderDate = new DateTime(2026, 2, 10), CustomerId = 2, StatusId = 2, IsActive = false, Total = 20 },
21             new Order { Id = 3, OrderDate = new DateTime(2026, 3, 10), CustomerId = 1, StatusId = 2, IsActive = true, Total = 30 });
22         await db.SaveChangesAsync();
23
24         var handler = new GetOrdersQueryHandler(db);
25
26         var unfiltered = await handler.Handle(new GetOrdersQuery { CustomerIds = [], StatusIds = [] }, CancellationToken.None);
27         Assert.Equal(3, unfiltered.Count);
28
29         var filtered = await handler.Handle(
30             new GetOrdersQuery
31             {
32                 DateFrom = new DateTime(2026, 2, 1),
33                 DateTo = new DateTime(2026, 12, 31),
34                 CustomerIds = [1, 2],
35                 StatusIds = [2],
36                 IsActive = true
37             },
38             CancellationToken.None);
39
40         var row = Assert.Single(filtered);
41         Assert.Equal(3, row.Id);
42     }
43 }
```

Unit test for Question 4

Question 5 — Process .cs files

CSharpFileProcessor in Wheelzy.Infrastructure walks a folder (and subfolders), skips bin / obj / .git, and uses Roslyn so renames hit real identifiers instead of random text.

Action	Behavior
a	Async methods whose name does not end with Async are renamed. Call sites are left unchanged, as requested.
b	Identifiers ending in Vm, Vms, Dto, Dtos become VM, VMs, DT0, DT0s.
c	Consecutive methods in the same type get a blank line between them if one is missing.

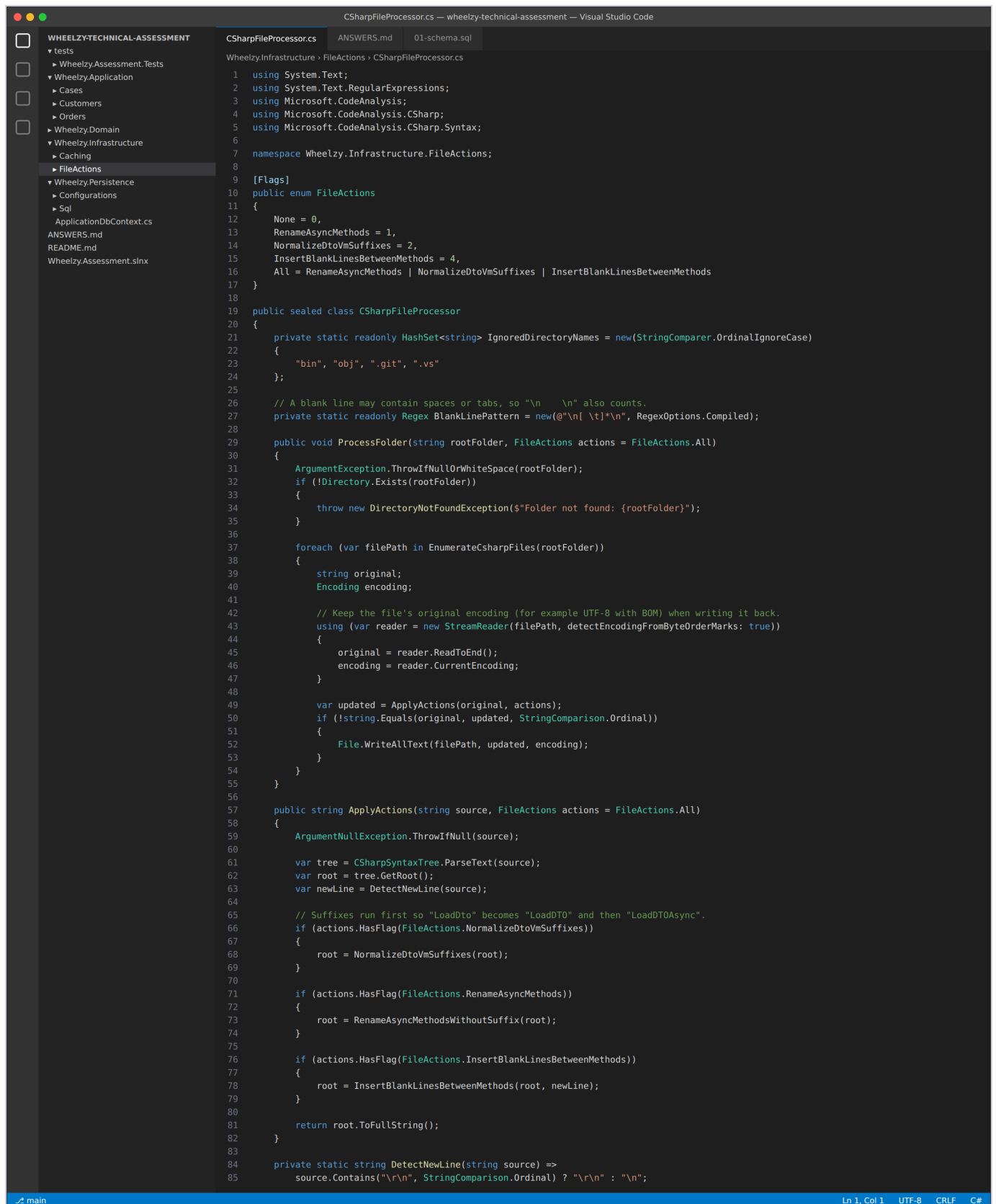
Robustness details:

- Suffixes are fixed before the Async rename, so LoadDto becomes LoadDT0Async (not LoadDtoAsync).
- The file's original line endings (CRLF / LF) and encoding (UTF-8 BOM) are kept.
- A line containing only spaces counts as a blank line; methods written on the same line are split correctly.

Tests in tests/Wheelzy.Assessment.Tests/CSharpFileProcessorTests.cs cover:

- async rename without updating references
- DTO / VM suffix rewrites
- inserting a missing blank line
- leaving an existing blank line alone
- processing nested folders with all three actions
- ignoring bin / obj folders
- keeping CRLF line endings and the UTF-8 BOM
- suffix-then-async ordering, whitespace-only blank lines, same-line methods
- not touching words where Dto/Vm is not at the end (DtoMapper, VmFactory)

Screenshots from the code



```
CSharpFileProcessor.cs — wheelzy-technical-assessment — Visual Studio Code

WHEELZY-TECHNICAL-ASSESSMENT
├── tests
├── Wheelzy.Assessment.Tests
├── Wheelzy.Application
├── Cases
├── Customers
├── Orders
├── Wheelzy.Domain
├── Wheelzy.Infrastructure
├── Caching
├── FileActions
├── Wheelzy.Persistence
├── Configurations
├── Sql
├── ApplicationDbContext.cs
├── ANSWERS.md
├── README.md
├── Wheelzy.Assessment.slnx
└── CSharpFileProcessor.cs

WHEELZY-INFRASTRUCTURE > FILEACTIONS > CSharpFileProcessor.cs

1  using System.Text;
2  using System.Text.RegularExpressions;
3  using Microsoft.CodeAnalysis;
4  using Microsoft.CodeAnalysis.CSharp;
5  using Microsoft.CodeAnalysis.CSharp.Syntax;
6
7  namespace Wheelzy.Infrastructure.FileActions;
8
9  [Flags]
10 public enum FileActions
11 {
12     None = 0,
13     RenameAsyncMethods = 1,
14     NormalizeDtoVmSuffixes = 2,
15     InsertBlankLinesBetweenMethods = 4,
16     All = RenameAsyncMethods | NormalizeDtoVmSuffixes | InsertBlankLinesBetweenMethods
17 }
18
19 public sealed class CSharpFileProcessor
20 {
21     private static readonly HashSet<string> IgnoredDirectoryNames = new(StringComparer.OrdinalIgnoreCase)
22     {
23         "bin", "obj", ".git", ".vs"
24     };
25
26     // A blank line may contain spaces or tabs, so "\n \n" also counts.
27     private static readonly Regex BlankLinePattern = new(@"\n[ \t]*\n", RegexOptions.Compiled);
28
29     public void ProcessFolder(string rootFolder, FileActions actions = FileActions.All)
30     {
31         ArgumentException.ThrowIfNullOrWhiteSpace(rootFolder);
32         if (!Directory.Exists(rootFolder))
33         {
34             throw new DirectoryNotFoundException($"Folder not found: {rootFolder}");
35         }
36
37         foreach (var filePath in EnumerateCsharpFiles(rootFolder))
38         {
39             string original;
40             Encoding encoding;
41
42             // Keep the file's original encoding (for example UTF-8 with BOM) when writing it back.
43             using (var reader = new StreamReader(filePath, detectEncodingFromByteOrderMarks: true))
44             {
45                 original = reader.ReadToEnd();
46                 encoding = reader.CurrentEncoding;
47             }
48
49             var updated = ApplyActions(original, actions);
50             if (!string.Equals(original, updated, StringComparison.Ordinal))
51             {
52                 File.WriteAllText(filePath, updated, encoding);
53             }
54         }
55     }
56
57     public string ApplyActions(string source, FileActions actions = FileActions.All)
58     {
59         ArgumentNullException.ThrowIfNull(source);
60
61         var tree = CSharpSyntaxTree.ParseText(source);
62         var root = tree.GetRoot();
63         var newLine = DetectNewLine(source);
64
65         // Suffixes run first so "LoadDto" becomes "LoadDTO" and then "LoadDTOAsync".
66         if (actions.HasFlag(FileActions.NormalizeDtoVmSuffixes))
67         {
68             root = NormalizeDtoVmSuffixes(root);
69         }
70
71         if (actions.HasFlag(FileActions.RenameAsyncMethods))
72         {
73             root = RenameAsyncMethodsWithoutSuffix(root);
74         }
75
76         if (actions.HasFlag(FileActions.InsertBlankLinesBetweenMethods))
77         {
78             root = InsertBlankLinesBetweenMethods(root, newLine);
79         }
80
81         return root.ToFullString();
82     }
83
84     private static string DetectNewLine(string source) =>
85         source.Contains("\r\n", StringComparison.Ordinal) ? "\r\n" : "\n";
```

CSharpFileProcessor.cs — folder walk and action pipeline

```
CSharpFileProcessor.cs — wheelzy-technical-assessment — Visual Studio Code

Wheelzy.Infrastructure > FileActions > CSharpFileProcessor.cs

86
87     private static IEnumerable<string> EnumerateCsharpFiles(string rootFolder)
88     {
89         return Directory.EnumerateFiles(rootFolder, "*.cs", SearchOption.AllDirectories)
90             .Where(path =>
91             {
92                 var relative = Path.GetRelativePath(rootFolder, path);
93                 return !relative.Split(Path.DirectorySeparatorChar, Path.AltDirectorySeparatorChar)
94                     .Any(segment => IgnoredDirectoryNames.Contains(segment));
95             });
96     }
97
98     private static SyntaxNode RenameAsyncMethodsWithoutSuffix(SyntaxNode root)
99     {
100         var methods = root.DescendantNodes()
101             .OfType<MethodDeclarationSyntax>()
102             .Where(method =>
103             {
104                 method.Modifiers.Any(SyntaxKind.AsyncKeyword) &&
105                 !method.Identifier.ValueText.EndsWith("Async", StringComparison.Ordinal));
106
107         return root.ReplaceNodes(methods, (_, method) =>
108         {
109             var newName = method.Identifier.ValueText + "Async";
110             return method.WithIdentifier(SyntaxFactory.Identifier(
111                 method.Identifier.LeadingTrivia,
112                 newName,
113                 method.Identifier.TrailingTrivia));
114         });
115     }
116
117     private static SyntaxNode NormalizeDtoVmSuffixes(SyntaxNode root)
118     {
119         return root.ReplaceTokens(
120             root.DescendantTokens().Where(token =>
121             {
122                 token.IsKind(SyntaxKind.IdentifierToken) && NeedsSuffixRewrite(token.ValueText));
123             (original, _) => SyntaxFactory.Identifier(
124                 original.LeadingTrivia,
125                 RewriteSuffix(original.ValueText),
126                 original.TrailingTrivia));
127     }
128
129     private static bool NeedsSuffixRewrite(string identifier) =>
130         !string.Equals(RewriteSuffix(identifier), identifier, StringComparison.Ordinal);
131
132     private static string RewriteSuffix(string identifier)
133     {
134         // Longest suffixes first so "Dtos" is not treated as "Dto" + "s".
135         if (identifier.EndsWith("Dtos", StringComparison.Ordinal))
136         {
137             return identifier[..^4] + "DTOS";
138         }
139
140         if (identifier.EndsWith("Dto", StringComparison.Ordinal))
141         {
142             return identifier[..^3] + "DTO";
143         }
144
145         if (identifier.EndsWith("Vms", StringComparison.Ordinal))
146         {
147             return identifier[..^3] + "VMS";
148         }
149
150         if (identifier.EndsWith("Vm", StringComparison.Ordinal))
151         {
152             return identifier[..^2] + "VM";
153         }
154
155         return identifier;
156     }
157
158     private static SyntaxNode InsertBlankLinesBetweenMethods(SyntaxNode root, string newLine)
159     {
160         return root.ReplaceNodes(
161             root.DescendantNodes().OfType<TypeDeclarationSyntax>(),
162             (_, type) => AddBlankLines(type, newLine));
163     }
```

CSharpFileProcessor.cs — async rename and DTO/VM suffixes

```
CSharpFileProcessor.cs — wheelzy-technical-assessment — Visual Studio Code
Wheelzy.Infrastructure > FileActions > CSharpFileProcessor.cs
160         (_, type) => AddBlankLines(type, newLine));
161     }
162
163     private static TypeDeclarationSyntax AddBlankLines(TypeDeclarationSyntax type, string newLine)
164     {
165         var members = type.Members;
166         if (members.Count < 2)
167         {
168             return type;
169         }
170
171         for (var i = 0; i < members.Count - 1; i++)
172         {
173             if (members[i] is not MethodDeclarationSyntax previous
174                 || members[i + 1] is not MethodDeclarationSyntax next)
175             {
176                 continue;
177             }
178
179             if (HasBlankLineBetween(previous, next))
180             {
181                 continue;
182             }
183
184             var previousEndsWithNewLine = previous.GetTrailingTrivia()
185                 .Any(trivia => trivia.IsKind(SyntaxKind.EndOfLineTrivia));
186
187             // If the previous method already ends its line, one extra line break makes a blank line.
188             // If both methods are on the same line, two line breaks are needed.
189             var breaksToInsert = previousEndsWithNewLine ? 1 : 2;
190             var leading = next.GetLeadingTrivia();
191             for (var n = 0; n < breaksToInsert; n++)
192             {
193                 leading = leading.Insert(0, SyntaxFactory.EndOfLine(newLine));
194             }
195
196             members = members.Replace(next, next.WithLeadingTrivia(leading));
197         }
198
199         return type.WithMembers(members);
200     }
201
202     private static bool HasBlankLineBetween(SyntaxNode first, SyntaxNode second)
203     {
204         var between = (first.GetTrailingTrivia().ToFullString() + second.GetLeadingTrivia().ToFullString())
205             .Replace("\r\n", "\n", StringComparison.Ordinal);
206         return BlankLinePattern.IsMatch(between);
207     }
208 }
```

CSharpFileProcessor.cs — blank lines between methods

WHEELZY-TECHNICAL-ASSESSMENT

tests

Wheelzy.Assessment.Tests

Wheelzy.Application

Cases

Customers

Orders

Wheelzy.Domain

Wheelzy.Infrastructure

Caching

FileActions

Wheelzy.Persistence

Configurations

Sql

ApplicationDbContext.cs

ANSWERS.md

README.md

Wheelzy.Assessment.slnx

CSharpFileProcessorTests.csANSWERS.md01-schema.sql

tests > Wheelzy.Assessment.Tests > CSharpFileProcessorTests.cs

```
1 using System.Text;
2 using Wheelzy.Infrastructure.FileActions;
3
4 namespace Wheelzy.Assessment.Tests;
5
6 public sealed class CSharpFileProcessorTests
7 {
8     private readonly CSharpFileProcessor _processor = new();
9
10    [Fact]
11    public void RenameAsyncMethods_AddsAsyncSuffixWithoutUpdatingReferences()
12    {
13        const string source = """
14            public class Sample
15            {
16                public async Task Save() { await Task.CompletedTask; }
17
18                public async Task SaveAsync() { await Task.CompletedTask; }
19
20                public void Call() { Save(); }
21            }
22            """;
23
24        var result = _processor.ApplyActions(source, FileActions.RenameAsyncMethods);
25
26        Assert.Contains("public async Task SaveAsync()", result);
27        Assert.Contains("public void Call() { Save(); }", result);
28        Assert.DoesNotContain("public async Task Save()", result);
29    }
30
31    [Fact]
32    public void NormalizeDtoVmSuffixes_RewritesWordsThatEndWithKnownSuffixes()
33    {
34        const string source = """
35            public class CustomerDto
36            {
37                public List<OrderDto> OrderDtos { get; set; }
38                public AccountVm AccountVm { get; set; }
39                public List<ItemVm> ItemVms { get; set; }
40            }
41            """;
42
43        var result = _processor.ApplyActions(source, FileActions.NormalizeDtoVmSuffixes);
44
45        Assert.Contains("public class CustomerDTO", result);
46        Assert.Contains("List<OrderDTO> OrderDTOs", result);
47        Assert.Contains("AccountVM AccountVM", result);
48        Assert.Contains("List<ItemVM> ItemVms", result);
49        Assert.DoesNotContain("CustomerDto", result);
50        Assert.DoesNotContain("OrderDtos", result);
51        Assert.DoesNotContain("ItemVms", result);
52    }
53
54    [Fact]
55    public void InsertBlankLines_AddsMissingLineBetweenConsecutiveMethods()
56    {
57        const string source = """
58            public class Sample
59            {
60                public void First() { }
61                public void Second() { }
62            }
63            """;
64
65        var result = _processor.ApplyActions(source, FileActions.InsertBlankLinesBetweenMethods);
66        var normalized = result.Replace("\r\n", "\n");
67
68        Assert.Contains("public void First() { }\n\n    public void Second() { }", normalized);
69    }
70
71    [Fact]
72    public void InsertBlankLines_LeavesExistingBlankLineAlone()
73    {
74        const string source = """
75            public class Sample
76            {
77                public void First() { }
78
79                public void Second() { }
80            }
81            """;
82
83        var result = _processor.ApplyActions(source, FileActions.InsertBlankLinesBetweenMethods);
84
85        Assert.Equal(source.Replace("\r\n", "\n"), result.Replace("\r\n", "\n"));
```

mainLn 1, Col 1UTF-8CRLFC#

CSharpFileProcessorTests.cs — core tests

WHEELZY-TECHNICAL-ASSESSMENT

tests

WheelzyAssessment.Tests

Wheelzy.Application

Cases

Customers

Orders

Wheelzy.Domain

Wheelzy.Infrastructure

Caching

FileActions

Wheelzy.Persistence

Configurations

Sql

ApplicationDbContext.cs

ANSWERS.md

README.md

WheelzyAssessment.slnx

CSharpFileProcessorTests.csANSWERS.md01-schema.sql

tests > WheelzyAssessment.Tests > CSharpFileProcessorTests.cs

```
140     }
141     """;
142
143     var result = _processor.ApplyActions(source, FileActions.All);
144
145     Assert.Contains("Task<OrderDTO> LoadDTOAsync()", result);
146 }
147
148 [Fact]
149 public void InsertBlankLines_KeepsWindowsLineEndings()
150 {
151     var source = "public class Sample\r\n{\r\n    public void First() { }\r\n    public void Second() { }\r\n}\r\n";
152
153     var result = _processor.ApplyActions(source, FileActions.InsertBlankLinesBetweenMethods);
154
155     Assert.Contains("public void First() { }\r\n\r\n    public void Second() { }", result);
156     Assert.DoesNotContain("\n", result.Replace("\r\n", string.Empty));
157 }
158
159 [Fact]
160 public void InsertBlankLines_SplitsMethodsWrittenOnTheSameLine()
161 {
162     const string source = "public class Sample { public void First() { } public void Second() { } }";
163
164     var result = _processor.ApplyActions(source, FileActions.InsertBlankLinesBetweenMethods);
165
166     Assert.Contains("public void First() { } \n\npublic void Second() { }", result);
167 }
168
169 [Fact]
170 public void InsertBlankLines_TreatsWhitespaceOnlyLineAsBlank()
171 {
172     const string source = "public class Sample\n{\n    public void First() { }\n    \n    public void Second() { } }\n";
173
174     var result = _processor.ApplyActions(source, FileActions.InsertBlankLinesBetweenMethods);
175
176     Assert.Equal(source, result);
177 }
178
179 [Fact]
180 public void ProcessFolder_SkipsBinAndObjFolders()
181 {
182     var root = Path.Combine(Path.GetTempPath(), "wheelzy-q5-" + Guid.NewGuid().ToString("N"));
183     var bin = Path.Combine(root, "bin");
184     Directory.CreateDirectory(bin);
185
186     const string content = "public class Skipped { public async Task Save() { await Task.CompletedTask; } }";
187     var target = Path.Combine(bin, "Skipped.cs");
188     File.WriteAllText(target, content);
189
190     try
191     {
192         _processor.ProcessFolder(root, FileActions.All);
193
194         Assert.Equal(content, File.ReadAllText(target));
195     }
196     finally
197     {
198         Directory.Delete(root, recursive: true);
199     }
200 }
201
202 [Fact]
203 public void ProcessFolder_KeepsUtf8Bom()
204 {
205     var root = Path.Combine(Path.GetTempPath(), "wheelzy-q5-" + Guid.NewGuid().ToString("N"));
206     Directory.CreateDirectory(root);
207
208     var target = Path.Combine(root, "Bom.cs");
209     File.WriteAllText(target, "public class ItemDto { }", new UTF8Encoding(encoderShouldEmitUTF8Identifier: true));
210
211     try
212     {
213         _processor.ProcessFolder(root, FileActions.All);
214
215         var bytes = File.ReadAllBytes(target);
216         Assert.True(bytes.Length > 3 && bytes[0] == 0xEF && bytes[1] == 0xBB && bytes[2] == 0xBF);
217         Assert.Contains("ItemDTO", File.ReadAllText(target));
218     }
219     finally
220     {
221         Directory.Delete(root, recursive: true);
222     }
223 }
224 }
```

mainLn 140, Col 1UTF-8CRLFC#

CSharpFileProcessorTests.cs — edge-case tests

Code Review — Is it senior level?

Verdict: yes, mostly. The structure and the choices are what a senior .NET developer would make. A few details needed fixing (listed below and already fixed in the repository).

Area	Rating	Notes
Architecture	Senior	Clean layering (Domain / Application / Persistence / Infrastructure), CQRS with MediatR, IApplicationDbContext abstraction, IEntityTypeConfiguration per entity.
Q1 – Database design	Senior	Normalized catalog, quote snapshot, filtered unique indexes for "only one current", trigger for the mandatory Picked Up date. Fixed: SQL was missing the BaseEntity audit columns that EF maps; EF query now uses LEFT JOINS instead of four correlated sub-queries.
Q2 – Caching	Senior	Covers the multi-instance problem (per-process cache, Redis, invalidation). Added: InvalidateAsync. Could still add stampede protection (lock per key) — not required.
Q3 – Balance update	Senior	Removes N+1, single save, null safety, grouping. Possible extras: report missing customer ids, and use a concurrency token (or ExecuteUpdateAsync) for concurrent balance changes.
Q4 – Dynamic filters	Senior	Composable IQueryable, AsNoTracking, DTO projection, cancellation token. Note: if dateTo is a date without time, callers should pass the end of the day.
Q5 – File processor	Mid → Senior after fixes	Using Roslyn instead of regex is a senior choice. Fixed: action order (Dto before Async), CRLF/BOM preserved, whitespace-only blank lines, double blank line insertion.
Tests	Good	xUnit + EF InMemory, 7 new edge-case tests added for Q5 and 1 for Q1 (case without current quote/status).

Before submitting: run `dotnet test Wheelzy.Assessment.slnx` locally — the new code was reviewed carefully but could not be compiled in my environment.

Corrections — Before / After

EF maps the BaseEntity audit columns, but the SQL tables did not have them, so saving through EF against SQL Server would fail with *Invalid column name*.

SaleCase / CaseQuote / CaseStatusHistory — SQL vs EF mapping

BEFORE

```
-- BEFORE: EF maps BaseEntity (CreatedBy, ModifiedOn, ModifiedBy,
-- IsActive, IsDeleted) but the SQL tables did not have these columns
CREATE TABLE dbo.SaleCase (
    CaseId      INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    Year        SMALLINT NOT NULL,
    SubmodelId  INT NOT NULL,
    ZipCode     CHAR(5) NOT NULL,
    CreatedAt   DATETIME2(0) NOT NULL DEFAULT (SYSUTCDATETIME())
);
-- => SaveChanges() fails: Invalid column name 'CreatedBy',
'IsActive'...
-- => CaseStatusHistory.CreatedOn was mapped to a column that did not
exist
```

AFTER

```
-- AFTER: the table matches the EF model
CREATE TABLE dbo.SaleCase (
    CaseId      INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    Year        SMALLINT NOT NULL,
    SubmodelId  INT NOT NULL,
    ZipCode     CHAR(5) NOT NULL,
    CreatedAt   DATETIME2(0) NOT NULL DEFAULT (SYSUTCDATETIME()),
    CreatedBy   NVARCHAR(256) NULL,
    ModifiedOn  DATETIME2(0) NULL,
    ModifiedBy  NVARCHAR(256) NULL,
    IsActive    BIT NOT NULL DEFAULT (1),
    IsDeleted   BIT NOT NULL DEFAULT (0)
);
// EF: CreatedOn -> "CreatedAt" + HasDefaultValueSql("SYSUTCDATETIME()")
// so a null CreatedOn uses the database default instead of inserting
NULL
```

The Async rename ran before the suffix rename, so LoadDto became LoadDtoAsync.

Question 5 — order of actions (a) and (b)

BEFORE

```
// BEFORE: Async rename ran first
root = RenameAsyncMethodsWithoutSuffix(root); // LoadDto ->
LoadDtoAsync
root = NormalizeDtoVmSuffixes(root);           // "LoadDtoAsync" does
not                                             // end with Dto ->
skipped
// Result: LoadDtoAsync (wrong)
```

AFTER

```
// AFTER: suffixes are fixed first
root = NormalizeDtoVmSuffixes(root);           // LoadDto -> LoadDTO
root = RenameAsyncMethodsWithoutSuffix(root); // LoadDTO ->
LoadDTOAsync
// Result: LoadDTOAsync (correct)
// Test: AllActions_RenameSuffixBeforeAddingAsync
```

Environment.NewLine mixed CRLF into LF files, and writing the file dropped its UTF-8 BOM.

Question 5 — line endings and file encoding

BEFORE

```
// BEFORE
SyntaxFactory.EndOfLine(Environment.NewLine)
// Windows adds "\r\n" into LF files (mixed line endings)

var original = File.ReadAllText(filePath);
File.WriteAllText(filePath, updated);
// UTF-8 BOM of the original file was dropped
```

AFTER

```
// AFTER
var newLine = source.Contains("\r\n") ? "\r\n" : "\n";
SyntaxFactory.EndOfLine(newLine) // same style as the file

using var reader = new StreamReader(filePath, true);
original = reader.ReadToEnd();
encoding = reader.CurrentEncoding;
File.WriteAllText(filePath, updated, encoding); // BOM kept
// Tests: InsertBlankLines_KeepsWindowsLineEndings,
//         ProcessFolder_KeepsUtf8Bom
```

A whitespace-only line was not treated as blank, and in some cases two blank lines were inserted.

Question 5 — detecting / inserting the blank line

BEFORE

```
// BEFORE
return between.Contains("\n\n");
// "\n  \n" (line with only spaces) was not seen as blank
// -> a second blank line was added

if (leading.Count == 0)
    insert TWO line breaks // even when the previous method
                           // already ended its line -> 2 blank lines
```

AFTER

```
// AFTER
Regex BlankLinePattern = new(@"\n[ \t]*\n");
return BlankLinePattern.IsMatch(between);

var previousEndsWithNewLine = previous.GetTrailingTrivia()
    .Any(t => t.IsKind(SyntaxKind.EndOfLineTrivia));
var breaksToInsert = previousEndsWithNewLine ? 1 : 2;
// Tests: InsertBlankLines_TreatsWhitespaceOnlyLineAsBlank,
//         InsertBlankLines_SplitsMethodsWrittenOnTheSameLine
```

Question 3: the original method from the assessment next to the improved version.

Question 3 — original method vs. improved method

BEFORE

```
// ORIGINAL (from the assessment)
foreach (var invoice in invoices)
{
    var customer = dbContext.Customers
        .SingleOrDefault(invoice.CustomerId.Value); // does not compile
    customer.Balance -= invoice.Total; // NullReference
    dbContext.SaveChanges(); // 1 save per
    invoice // + N+1 queries
}
```

AFTER

```
// IMPROVED
var totals = invoices.Where(i => i.CustomerId.HasValue)
    .GroupBy(i => i.CustomerId!.Value)
    .ToDictionary(g => g.Key, g => g.Sum(i => i.Total));

var ids = totals.Keys.ToList();
var customers = await db.Customers
    .Where(c => ids.Contains(c.Id)).ToListAsync(ct); // 1 query

foreach (var c in customers) c.Balance -= totals[c.Id];
await db.SaveChangesAsync(ct); // 1 save
```